

When Does Test-Time Search Help LLMs Classify Vulnerabilities?

A Compute-Matched Study of MCTS over Taxonomy and Reasoning

Anonymous Authors

Abstract

Test-time search — self-consistency, best-of- N , beam search, and Monte Carlo Tree Search (MCTS) — is increasingly applied to make large language models (LLMs) reason more reliably. Yet reported gains often compare a search method running many model calls against a single greedy call, conflating the value of *search* with the value of *extra compute*. It remains unclear not only *when* structured search helps an LLM, but *what it should search over*. We present a controlled, **compute-matched** study of test-time search for hierarchical vulnerability classification over the Common Weakness Enumeration (CWE) taxonomy. Holding the policy LLM, prompt, and token budget fixed, we compare search strategies and isolate three axes the literature entangles: *policy strength*, *reward fidelity* (self-evaluation vs. a stronger external judge), and—most consequentially—the *search space*, contrasting MCTS over the output taxonomy against MCTS over a free-form reasoning trajectory. Three findings on a code-vulnerability benchmark (BigVul, patch-diff inputs). (i) *What you search over dominates*: MCTS over a reasoning trajectory beats MCTS over the taxonomy by +12 leaf-accuracy points at *half* the token budget ($p < 10^{-4}$), because taxonomy search commits to a wrong coarse category on 60% of examples and cannot recover, whereas reasoning search defers commitment; moreover its accuracy is flat in the iteration knob, so the gain comes from the reasoning *scaffold* rather than the search machinery. (ii) Search compensates *weak* policies (up to +8 points over greedy) but not strong ones, and its gain shrinks monotonically with policy strength; self-consistency (sampling without a reward) captures almost none of it. (iii) A stronger external judge leaves taxonomy search unchanged and does not survive budget matching on reasoning search—the bottleneck is the policy and the search space, not reward fidelity. We conclude that the design of the *search space*, not the search budget or the reward, is the primary lever for test-time search on structured outputs.

1 Introduction

Test-time search has become a default recipe for coaxing more reliable reasoning out of large language models (LLMs): sample several chains and vote (self-consistency), score samples and keep the best (best-of- N), expand a beam, or run Monte Carlo Tree Search (MCTS) over intermediate steps. Reported gains, however, are frequently confounded: a search method issuing dozens of model calls is compared against a single greedy call, so it is unclear whether the improvement comes from *search structure* or simply from *spending more compute*.

We study this question on *hierarchical vulnerability classification*: given a code fragment, predict a root-to-leaf path in the CWE taxonomy, at the operational granularity used to label real CVEs (§3). This task is a good testbed because (i) it has a real, structured output space with a natural search tree; (ii) errors compound across levels, so search has a plausible mechanism to help *or* to hurt; and (iii) public, CWE-labeled datasets exist.

```

// gold: CWE-416 (Use After Free)
int new_high = tree[current].high;
tree[current].high = low-1;
add_range(ctx, cmap, high+1, new_high, ...);
+ tree = cmap->tree;    // fix: re-fetch; add_range may realloc
}
if (tree[current].low > high) {

```

Figure 1: A motivating pilot example (ghostscript; abridged). `add_range` may reallocate `cmap->tree`, so the subsequent reads through the stale `tree` pointer are a use-after-free (CWE-416). Greedy answers CWE-125 (out-of-bounds *read*); self-consistency at $8\times$ the budget agrees with greedy; MCTS over the taxonomy spends 10.7k tokens, commits at the first level to the *spatial* out-of-bounds category, and lands on CWE-787 — the wrong branch, unrecoverable. MCTS over a *reasoning trajectory* notices the realloc-then-reuse pattern and answers CWE-416 at a comparable budget. More compute did not help; searching a different space did.

The compute confound. Our central methodological commitment is **compute-matched comparison**: every strategy is evaluated across a grid of token budgets $\mathcal{B}$, and each method’s knob (N , beam width, or MCTS iterations) is tuned so its expected token spend matches $\mathcal{B}$. The headline result is an accuracy-vs-budget curve, not a single “MCTS beats greedy” number.

Three axes we isolate. Whether and how search helps plausibly depends on (a) the base policy’s strength, (b) the fidelity of the reward guiding it, and (c)—we argue, most importantly—the *space* the search operates in. We vary *policy strength* $\alpha(\pi)$ across LLMs, contrast self-evaluation against a stronger external judge, and compare searching the output *taxonomy* against searching a free-form *reasoning trajectory*.

Contributions.

1. Our central result: **what you search over dominates how much you search**. MCTS over a reasoning trajectory beats MCTS over the taxonomy by +12 leaf-accuracy points at *half* the token budget ($p < 10^{-4}$), and its accuracy is flat in the iteration knob—the reasoning *scaffold*, not the search machinery, carries the gain. A cascade analysis shows why: taxonomy search commits to a wrong coarse category on 60% of examples and cannot recover, whereas reasoning search defers commitment (§6).
2. A **compute-matched protocol** for evaluating test-time search on a structured-output task, removing the search-vs-compute confound, with a controlled comparison of strategies (greedy, self-consistency, best-of- N , beam, MCTS-Tax, MCTS-Reason) on public CWE-labeled data (§5).
3. The marginal gain of search *decreases* with policy strength: large for weak policies, negligible-to-negative for strong ones—test-time search compensates for weak policies rather than improving strong ones (§6).
4. A stronger external-judge reward does not improve search, locating the bottleneck in the policy and the search structure rather than reward fidelity (§6).

We emphasize that this is a *diagnostic* study: a finding that MCTS does not beat cheaper search at matched budget is a valid and useful result, and we design the paper so its contribution does not depend on the sign of that result.

2 Related Work

Test-time scaling and search for LLMs. Sampling several chains of thought and voting (self-consistency) [26] is the simplest way to spend more inference compute; best-of- N adds a verifier that selects among samples [7], and repeated sampling alone scales surprisingly far [2]. Structured alternatives organize generation as explicit search: Tree-of-Thoughts [30], MCTS-style planning over reasoning states [14], agentic tree search [33], and a family of MCTS-for-reasoning systems such as rStar [20], AlphaMath [4], and ReST-MCTS* [31]. Studies of compute-optimal inference [22, 29] show that the best strategy depends on the difficulty of the problem and the strength of the base model. Most system papers in this family, however, report a search method issuing dozens of calls against a single greedy call, leaving the contribution of search *structure* (as opposed to extra compute) unidentified.

Compute-matched comparison of inference strategies. A smaller line of work controls for compute explicitly. Chen et al. [5] analyze when more LLM calls help or hurt; Olausson et al. [19] show that self-repair for code loses much of its apparent advantage once its extra calls are budget-matched against plain sampling; Snell et al. [22] compare revisions, best-of- N , and tree search per unit of compute; and Stroebel et al. [23] show that resampling against an imperfect verifier saturates and then degrades. We bring this discipline to a structured-output security task, and additionally match budgets *across search spaces* (taxonomy vs. reasoning trajectory), an axis these works do not vary.

Process rewards, verifiers, and self-evaluation. Search needs a value signal. Trained process reward models outperform outcome supervision on math [18, 25], but require labeled step data we deliberately exclude (§5); we instead use the two zero-shot signals available in practice: the policy’s own self-evaluation and a stronger external LLM judge [32]. Both are known to be imperfect: LLMs are unreliable judges of their own reasoning [16], and imperfect verifiers cap the benefit of more search [23]. Our RQ3 quantifies exactly this: whether upgrading reward fidelity moves accuracy at all on our task (it does not).

LLMs for vulnerability detection and CWE classification. The encoder era trained graph- and token-based classifiers on labeled functions — Devign [35], ReVeal [3], LineVul [12] — mostly as *binary* vulnerable/benign detection. PrimeVul [9] showed that label noise and leakage inflate these results and that code LLMs remain far from reliable detectors; DiverseVul [6], BigVul [11], and CVEfixes [1] provide the CWE-labeled corpora we build on. LLM-era systems add knowledge and agency — LLM4Vuln [24], Vul-RAG [10], GPTLens [15] — but still predominantly target detection; fine-grained CWE assignment is usually a flat multi-class afterthought, with V2W-BERT [8] a notable hierarchical exception in the pre-LLM setting. Test-time *search* for CWE classification under controlled compute has, to our knowledge, not been studied.

Hierarchical classification. Hierarchical text classification has a long tradition of exploiting the label tree, from the taxonomy of approaches in Silla and Freitas [21] to neural methods such as HMCN [28], HiAGM [34], and HGCLR [27], with dedicated hierarchical metrics [17]. The CWE tree gives MCTS-TAX its natural search space; our cascade analysis (§7) echoes the classic error-propagation argument against greedy top-down classification, now in the test-time-search setting.

Positioning. Unlike prior MCTS-for-LLM work, we (a) compare strategies *at matched token budget* rather than matched iterations, (b) explicitly disentangle policy strength from reward

fidelity, and (c) contrast search over the output taxonomy against search over a reasoning trajectory — on a public, reproducible CWE-classification benchmark.

3 Problem Formulation

Taxonomy and labels. Let $\mathcal{T} = (V, E)$ be the CWE taxonomy: a directed tree (with a small number of cross-edges resolved to a primary parent) rooted at r , where each node $v \in V$ has a textual description $\delta(v)$. A labeled example is a pair (c, p) where c is a code fragment and $p = (v_0, v_1, \dots, v_k)$ is a root-to-leaf path with $v_0 = r$; write $\text{leaf}(p) = v_k$.

Granularity regime. CWE ships several views of different granularity. The full Research-Concepts view (View-1000, 716 leaves) is designed for weakness research; MITRE’s own mapping guidance directs CVE labeling to the much smaller View-1003 (“Weaknesses for Simplified Mapping of Published Vulnerabilities”, 130 CWEs), which is what NVD uses in practice. We study the *operational* regime: our label space (7 code-aligned categories over 26 CWE leaves, Appendix A) is a subset of View-1003 covering the labels that occur in the data. The full-depth view serves as a boundary condition: we show in §7 that at View-1000 granularity every current policy — including the premium one — sits at the signal floor, where the question “does search help” degenerates.

Hierarchical classification. A predictor f maps a code fragment c to a path $\hat{p}$. We report leaf exact-match accuracy, path accuracy@ k , pillar (level-0) accuracy, hierarchical F_1 on path edges [17], and Wu–Palmer similarity between $\hat{p}$ and p .

Policy and test-time strategies. Fix a base *policy* LLM π and a per-step prompt template. A *test-time strategy* X is a procedure that, given c , issues one or more calls to π (optionally guided by a reward function r) and returns a path. We study the family

$$X \in \{\text{Flat}, \text{Greedy}, \text{SelfConsistency}, \text{BestOfN}, \text{Beam}, \text{MCTS-TAX}, \text{MCTS-REASON}\}.$$

Each non-trivial strategy has a knob (e.g. N , beam width w , MCTS iterations I) controlling how much it searches.

Compute budget. Let $\text{tok}_X(c)$ be the total tokens (input+output) consumed by strategy X on c , and $\mathcal{B}$ a target per-example budget. We say two strategies are *compute-matched at $\mathcal{B}$* if their expected token spend equals $\mathcal{B}$. The central object of study is the budget-conditioned accuracy

$$\text{Acc}_X(\mathcal{B}) = \mathbb{E}_c[\mathbf{1}[\hat{p}_X(c; \mathcal{B}) = p(c)]],$$

where X ’s knob is set so that $\mathbb{E}_c[\text{tok}_X(c)] = \mathcal{B}$.

Policy strength and marginal search gain. Let $\alpha(\pi)$ denote the pillar (level-0) accuracy of greedy decoding under policy π ; this is our scalar proxy for policy strength. The *marginal search gain* of a strategy at budget $\mathcal{B}$ is

$$\Delta_X(\pi, \mathcal{B}) = \text{Acc}_X(\mathcal{B}) - \text{Acc}_{\text{Greedy}}.$$

Assumption 1 (Description availability). The textual description $\delta(v)$ is available for every node $v \in V$ at inference time. This reflects the public MITRE CWE taxonomy and lets every strategy condition on candidate descriptions without taxonomy-specific training.

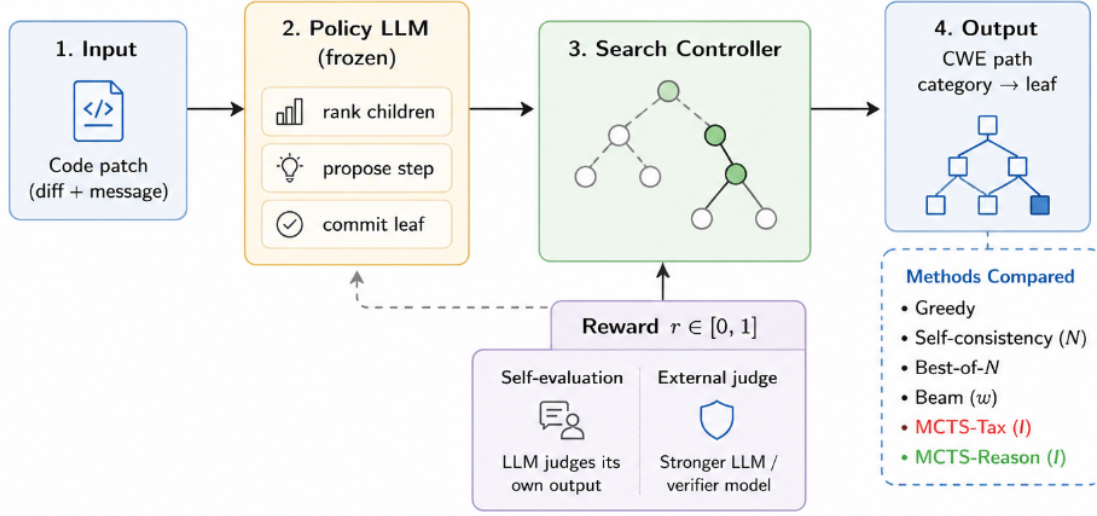


Figure 2: Shared architecture. Every strategy drives the same frozen policy LLM through the same three operations and differs only in the search controller it plugs in; the reward signal (self-evaluation or external judge) is used only by the reward-guided strategies (best-of- N , beam, MCTS).

Proposition 2 (Cascade compounding). *Suppose hierarchical decoding makes an independent error with probability $1 - \alpha$ at each of k levels. Then path accuracy is at most α^k , so the headroom a search strategy can recover, $1 - \alpha^k$, is largest when α is small. A reward-guided search converts this headroom into accuracy only to the extent that its reward can identify a better branch than the greedy choice; we find empirically (§6) that this payoff is largest for weak policies and shrinks as α approaches the task’s accuracy ceiling.*

Proposition 2 is the informal mechanism behind hypotheses H2 and H4 (§5): search interacts with policy strength, and taxonomy search (which commits level-by-level) is more exposed to early-level compounding than reasoning-trajectory search.

4 Methods: A Ladder of Test-Time Search

All strategies share one policy LLM π , one per-step prompt template (contrastive: candidate CWE descriptions are shown side by side), and one structured-output schema. The *only* thing that varies across strategies is how they search (Figure 2). This is what makes the compute-matched comparison (§5) a clean measurement of search structure rather than prompt engineering.

4.1 No-search baselines

Flat (M0a): a single call lists all leaf candidates and the model picks one. **Greedy-hierarchical (M0b):** one call per level, taking the argmax child; defines $\text{Acc}_{\text{Greedy}}$ and the policy-strength proxy $\alpha(\pi)$.

4.2 Sampling strategies

Self-Consistency (M1): sample N independent paths at temperature $T > 0$, return the majority leaf. **Best-of- N (M2):** sample N paths, return the one maximizing the reward r (§4.4). M1 isolates the value of sampling alone; M2 adds the value of a reward signal.

	Strategy	Searches over	Knob
M0a	Flat	— (one call, all leaves)	—
M0b	Greedy	— (argmax per level)	—
M1	Self-consistency	i.i.d. samples, majority	N
M2	Best-of- N	i.i.d. samples, reward-max	N
M3	Beam	taxonomy, width-limited	w
M4	MCTS-TAX	taxonomy, reward-guided	I
M5	MCTS-REASON	reasoning steps, reward-guided	I

Table 1: The ladder of strategies, ordered by increasing search structure. All share one policy, prompt, and output schema; each strategy’s knob is set by the compute-matched protocol (§5).

Algorithm 1 MCTS-TAX: MCTS over the CWE taxonomy

Require: code c , taxonomy $\mathcal{T}$, policy π , reward r , budget I iterations

- 1: root state $s_0 \leftarrow (r)$ $\triangleright$ partial path
 - 2: **for** $i = 1, \dots, I$ **do**
 - 3: **Select:** descend by $Q(s, a) + c_{\text{exp}} P(a) \sqrt{N(s) + 1} / (1 + N(s, a)) \triangleright$ PUCT; P from policy scores
 - 4: **Expand:** π scores the children of the selected node; scores become priors P
 - 5: **Evaluate:** $r(s')$ scores the (partial/complete) path via self-eval or judge
 - 6: **Backprop:** update Q, N up the search tree
 - 7: **end for**
 - 8: **return** path through most-visited children (robust child)
-

4.3 Structured search

Beam (M3): maintain the top- w partial paths per level under r ; deterministic given π .
MCTS-Tax (M4): MCTS over the taxonomy.

MCTS-Reason (M5): the same selection/backup core, but search-tree nodes are *reasoning states* (a sequence of short free-form reasoning steps about the patch), not CWE nodes. Each move either appends one reasoning step or *commits* to a single leaf CWE from the flat leaf list (prompts in Appendix B); rollouts force a commit and score the resulting root-to-leaf path, and the committed leaf is deterministically mapped to a taxonomy path. Crucially, M5 never commits level-by-level: the coarse category is implied by the final leaf, not chosen first. M4 vs. M5 is our test of whether search over the output structure or over reasoning is more robust to early-level compounding (Proposition 2).

4.4 Reward signal

We deliberately use *only* zero-shot reward sources, with no trained process reward model (PRM), to keep the study GPU-free and reproducible:

- **Self-eval:** the policy π scores its own (partial) path in $[0, 1]$.
- **External judge:** a separate, typically stronger model scores the path given c and candidate CWE descriptions. We record the pair (π, judge) .

Comparing self-eval vs. judge directly probes whether the bottleneck is the policy or the reward (RQ3/H3). A trained PRM is left to future work (§8).

4.5 Structured outputs

Each policy call returns JSON — per-candidate scores plus a one-sentence rationale that is carried into the next level’s prompt — parsed defensively with a small floor score for missing

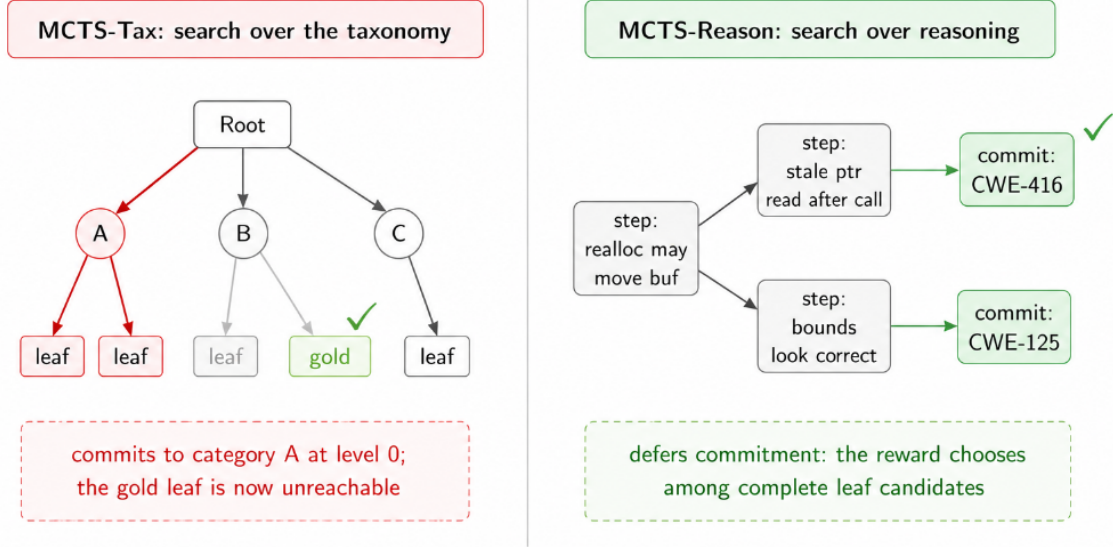


Figure 3: The two MCTS search spaces. MCTS-TAX (left) walks the label tree and must pick a coarse category before it has reasoned about the patch; a wrong first move makes the gold leaf unreachable. MCTS-REASON (right) searches over short free-form reasoning steps and commits to a leaf directly, so the category is implied by the final choice rather than locked in first.

candidates, so malformed outputs never crash the search (Appendix B). Conformal calibration of path-level prediction sets from the score vectors is part of the final protocol.

5 Experimental Setup

Research questions and hypotheses. **RQ1** (compute-matched): does MCTS exceed cheaper search at equal token budget? **RQ2** (policy): how does the marginal search gain Δ depend on policy strength $\alpha(\pi)$? **RQ3** (reward): is the bottleneck the policy or the reward? **RQ4** (search space): taxonomy vs. reasoning search? The corresponding hypotheses are **H1** $\text{Acc}_{\text{MCTS-TAX}}(\mathcal{B}) - \text{Acc}_{\text{SC}}(\mathcal{B}) \leq \epsilon$ for weak policies; **H2** Δ increases in $\alpha(\pi)$ with a threshold α^* below which $\Delta \leq 0$; **H3** switching self-eval $\rightarrow$ judge moves leaf accuracy by $\ll$ the gain from a stronger policy; **H4** reasoning search beats taxonomy search for weak policies.

5.1 Datasets and Splits

We assemble CWE-labeled data from four public sources — PrimeVul, DiverseVul, BigVul, and CVEFixes — and deduplicate at the function level (MinHash, Jaccard ≥ 0.85) and at the repository level across all sources and splits to prevent leakage. The CWE tree is built from MITRE relationships (pillars + depth 3, paths deduplicated) and released with a SHA-256 hash. Splits are *stratified* train / calibration / test (target test size ≈ 2000 ; dev set 50–100); a temporal split by CVE publication date is reported as a robustness check (§7).

Pretraining-leakage control. Because LLMs may have seen public CVEs during pretraining, we report accuracy on the subset whose CVE was published *after* each model’s cutoff as a fair slice, alongside the full set.

Pilot-scale data statistics (sources, splits, per-category composition, tree hashes) are given in Appendix A; the final-protocol dataset table will be added with the headline runs.

Strategy	nano (weak)		Haiku (strong)	
	Leaf	Cat.	Leaf	Cat.
Greedy (no search)	14.7	28.7	24.7	46.7
Self-Consistency ($N=8$)	16.5	31.5	24.0	46.7
MCTS-TAX ($I=16$)	22.9	39.5	26.6	44.4
Δ (MCTS-TAX – greedy)	+8.2	+10.8	+1.9	−2.3

Table 2: Compute-matched test-time search (accuracy %), weak vs. strong policy (nano $n=300/5$ seeds; Haiku $n=150/3$ seeds, greedy and SC deterministic or 1 seed). At equal budget, MCTS-TAX lifts the weak policy by +8.2 (leaf) / +10.8 (category) points, but the strong policy by only +1.9 / −2.3. Self-consistency helps neither. The marginal search gain Δ *shrinks* as policy strength grows.

5.2 Strategies and Models

The seven strategies of §4 all use the same policy and prompt. We use four cheap-tier policies spanning a range of strength (GPT-4.1-nano, Gemini-2.5-Flash-Lite, DeepSeek-V3.1, Claude-Haiku-4.5) for the pilot, plus a premium policy (Claude-Sonnet-4.6) on a reduced sample to probe the strong end of the $\alpha(\pi)$ axis required for H2; the headline phase extends the premium coverage. At least two open-weight models (DeepSeek + Qwen2.5-Coder-32B or Llama-3.3-70B) are included for reproducibility. The external judge is a model stronger than the current policy; each (π, judge) pair is logged.

5.3 Compute-Matched Protocol

We first measure $\text{tok}_X(c)$ per knob value on the dev set, then define a budget grid $\mathcal{B} \in \{1\times, 2\times, 4\times, 8\times, 16\times, 32\times\}$ relative to a single greedy pass. For each $\mathcal{B}$ we set every strategy’s knob to match expected tokens, and compare strategies *only* at equal $\mathcal{B}$. We additionally report wall-clock latency (P50/P95): self-consistency parallelizes trivially, MCTS does not — a practically important asymmetry.

5.4 Metrics and Statistics

Primary: leaf accuracy, pillar accuracy, hierarchical- F_1 , and the accuracy-vs-budget curve $\text{Acc}_X(\mathcal{B})$. Secondary: path accuracy@ k , Wu–Palmer, ECE, tokens/sample, \$/sample, latency. All configurations use 5 seeds; we report mean $\pm$ std and 95% bootstrap CIs (10^4 resamples). Strategy comparisons at fixed $\mathcal{B}$ use paired McNemar with Bonferroni correction.

6 Results

6.1 RQ1: Search vs. Compute (the headline)

Table 2 reports the compute-matched comparison on a *weak* policy (GPT-4.1-nano) and a *strong* policy (Claude-Haiku-4.5), on BigVul examples (diff+commit-message input, 7-category code-aligned tree), comparing greedy, self-consistency, and MCTS-TAX at a token budget where the latter two spend roughly equally within each policy. (*Pilot: nano $n=300/5$ seeds, Haiku $n=150/3$ seeds; the final protocol scales to $n\approx 2000$ with paired McNemar throughout, see §5.*)

Finding. (i) On the weak policy, at matched compute, reward-guided tree search clearly beats both no-search and sampling-without-reward: MCTS-TAX reaches 22.9% leaf accuracy versus 16.5% (self-consistency) and 14.7% (greedy); across-seed std is ≤ 1.0 . Both gaps are significant by paired McNemar (MCTS-TAX vs. greedy $p=8\times 10^{-6}$; vs. self-consistency $p=2\times 10^{-3}$, $n=300$).

Self-consistency—majority vote over independent samples, ignoring the reward—captures only +1.8 points, whereas MCTS, using the same self-evaluation reward to *steer* the search, captures +8.2. Best-of- N ($N=8$), which keeps the reward but drops the tree (independent samples, return the highest-scoring path), lands in between: +6.4 leaf points at a *larger* budget (15.8k vs. 10.7k tokens). Reading the three together: the reward signal alone recovers most of the sampling family’s gap to tree search, and the tree adds the rest while spending less. (ii) On the strong policy the same search yields little (+1.9 leaf) to nothing (−2.3 category): the strong policy is already near the signal available in the input, so reward-guided exploration mostly reconfirms or drifts. Test-time search here acts as a *compensator for weak policies*, not a universal improvement.

6.2 RQ2: Search Gain vs. Policy Strength

Across three policy points the marginal search gain Δ of MCTS-TAX over greedy (category accuracy) *decreases monotonically* with policy strength $\alpha(\pi)$:

Policy $\alpha(\pi)$ (greedy cat.)	Gemini-FL 0.28	nano 0.29	DeepSeek 0.37	Haiku 0.47	Sonnet 0.48
Δ (MCTS-TAX, cat.)	+6.7	+10.8	+2.0	−2.3	+1.7
Δ (MCTS-REASON, cat.)	+14.2	+19.4	+8.2	−3.8	+2.5

The taxonomy-search gain falls from +7–11 points at the weak end ($\alpha(\pi) \approx 0.28$, two independent policies) to $\approx 0 \pm 3$ at the strong end (Haiku −2.3, Sonnet +1.7); the reasoning-search gain starts far higher (+14–19) and collapses onto the same near-zero band (−3.8 / +2.5). At the strong end the residual gains are within pilot noise of zero.

This **contradicts our pre-registered H2**, which anticipated that gain would *increase* with $\alpha(\pi)$ (a threshold α^* below which search fails). The data show the opposite ordering: search is most valuable exactly where the policy is weakest, and vanishes as the policy approaches the accuracy ceiling imposed by the task/labels (§7). We therefore revise the claim: test-time search trades compute for accuracy on under-powered policies, and offers diminishing-to-negative returns on strong ones.

One nuance: on the mid-strength policy (DeepSeek), plain self-consistency ($N=8$: 28.2 leaf) actually edges out MCTS-TAX (25.3) while both trail MCTS-REASON (27.8 leaf, 44.9 category) — so the pre-registered H1 (MCTS-TAX within ϵ of self-consistency at matched budget) fails for the weak policy but roughly holds mid-scale. The ordering that is stable across all three policies is the search-space one (RQ4).

(Pilot: nano $n=300/5$ seeds; DeepSeek and Haiku $n=130-150/3$ seeds on the same example prefix; greedy is deterministic. The final protocol runs all policies on identical examples with bootstrap CIs and adds a premium point.)

6.3 RQ3: Is the Reward the Bottleneck?

If reward fidelity limited MCTS, replacing the policy’s self-evaluation with a stronger external judge (Claude-Haiku-4.5, the strong policy of RQ1, judging the weak nano policy) should help. On matched examples ($n=300$, 3 seeds): for MCTS-TAX ($I=16$) the judge changes nothing — 22.8 ± 0.5 leaf accuracy vs. 22.9 ± 0.1 with self-evaluation — while costing 71% more tokens (18.3k vs. 10.7k). For MCTS-REASON ($I=8$) the judge yields a small gain (35.4 ± 0.6 vs. 33.9 ± 0.3) at 27% more tokens; but self-eval MCTS-REASON at $I=2$ already reaches 34.8 ± 0.4 at less than a *third* of the judge configuration’s budget, so the gain does not survive compute matching.

Finding. Upgrading reward fidelity does not move taxonomy search at all — its early-commitment pathology is structural, and no reward source can fix a search-space problem — and buys, on reasoning search, less than its extra tokens are worth elsewhere. The reward source

Strategy	nano (weak)		Haiku (strong)	
	Leaf	Cat.	Leaf	Cat.
Greedy	14.7	28.7	24.7	46.7
MCTS-TAX ($I=16$)	22.9	39.5	26.6	44.4
MCTS-REASON ($I=8$)	33.9	48.1	28.4	42.9

Table 3: Taxonomy vs. reasoning search (accuracy %; nano $n=300/5$ seeds, Haiku $n=150/3$ seeds). Reasoning search beats taxonomy search on *both* policies; the margin is large for the weak policy (+11 leaf) and modest for the strong one (+1.8).

is therefore not the binding constraint; the policy and the search space are (consistent with RQ2 and RQ4). (*A reward-calibration analysis — AUROC of reward vs. path correctness for self-eval and judge — is part of the final protocol.*)

6.4 RQ4: Taxonomy vs. Reasoning Search

We compare MCTS over the taxonomy (MCTS-TAX, committing level-by-level) against MCTS over a free-form reasoning trajectory (MCTS-REASON, which reasons and then commits directly to a leaf), at a comparable budget on the weak policy (Table 3, $n=150$).

Finding. Reasoning-trajectory search dominates taxonomy search on *both* policies (weak: 33.9 vs. 22.9 leaf, +11, paired McNemar $p=3.2 \times 10^{-4}$, $n=300$; strong: 28.4 vs. 26.6, +1.8), with the advantage shrinking as the policy strengthens (cf. RQ2). On the strong policy it trades a small category drop (42.9 vs. greedy 46.7) for a leaf gain, reaching finer leaves at the cost of occasionally landing in a neighbouring category. On the premium policy (Sonnet, $n=120$, 1 seed) the ordering *reverses* on leaf accuracy (31.7 vs. 35.0 for MCTS-TAX): a policy that already reasons well in a single pass gains nothing from an external reasoning scaffold, while the taxonomy structure still adds a little fine-grained discrimination. The search-space advantage, like the search gain itself, is a property of *weak-to-mid* policies.

Strict budget matching. Sweeping MCTS-REASON over $I \in \{2, 4, 6, 8\}$ ($n=300$, 5 seeds, Figure 4) shows its accuracy is *flat in the iteration count*: $I=2$, at 5.3k tokens/example — *half* of MCTS-TAX’s 10.7k budget — already reaches 34.8% leaf / 50.4% category accuracy, slightly *above* $I=8$ (33.9 / 48.1). The dominance over taxonomy search therefore survives strict budget matching with room to spare (MCTS-REASON at $I=2$ vs. MCTS-TAX at $I=16$: paired McNemar $p=8.7 \times 10^{-5}$; vs. greedy: $p=4.4 \times 10^{-10}$). The flatness is itself informative: nearly all of MCTS-REASON’s gain comes from its *representation* — reason first, commit to a leaf directly — rather than from accumulating search statistics; we return to this in §7.

The mechanism is the cascade (Proposition 2): MCTS-TAX commits to a category at level 0 and cannot recover the correct leaf once that choice is wrong, whereas MCTS-REASON defers commitment, reasons over the patch, and selects a leaf directly—side-stepping early-level lock-in. The error breakdown (5 seeds $\times$ 300 examples, Figure 5) confirms this: MCTS-TAX commits to a wrong category on 60.5% of examples (unrecoverable by construction) versus 51.9% for MCTS-REASON, and on the cases where MCTS-TAX picks the wrong category, MCTS-REASON recovers the correct one 25.7% of the time. Strikingly, the weak policy with reasoning search reaches category accuracy (50.4% at $I=2$) *above* the strong policy’s greedy accuracy (46.7%, Table 2).

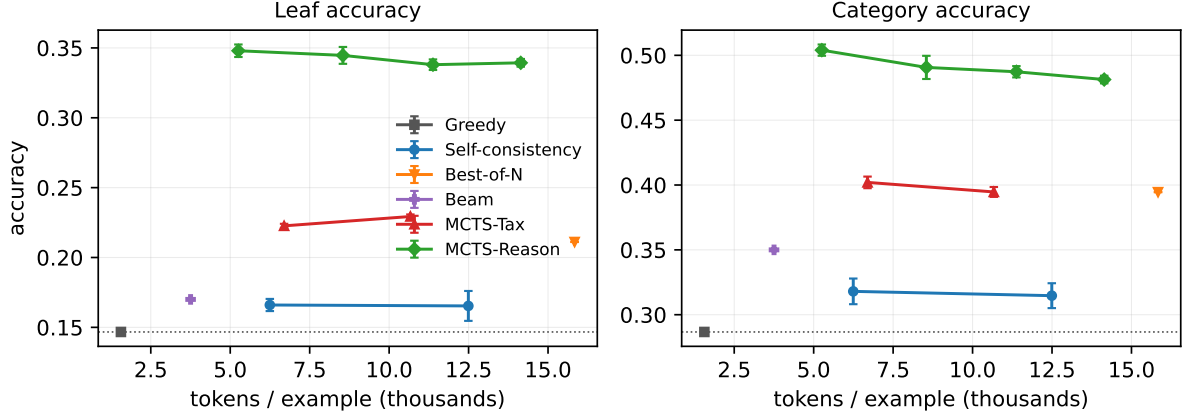


Figure 4: Accuracy vs. token budget (GPT-4.1-nano, $n=300$, mean $\pm$ std over 5 seeds; greedy, beam, and best-of- N are single runs). MCTS-REASON dominates at every budget and is flat in its iteration knob; MCTS-TAX saturates between $I=8$ and $I=16$; best-of- N (reward, no tree) sits between self-consistency and MCTS-TAX; self-consistency barely improves on greedy. What is searched matters far more than how much.

7 Analysis

7.1 When does search pay off?

Synthesizing RQ1–RQ4 yields a simple decision rule for test-time search on this structured-output task:

1. **Search the reasoning, not the taxonomy — for weak-to-mid policies.** MCTS-REASON beats MCTS-TAX on four of five policies (Table 3), and on the weak policy even at *half* the budget (Figure 4); committing to coarse categories early is the single most costly design choice. On the premium policy the two are within noise of each other and of greedy.
2. **Spend on search when the policy is weak.** The marginal gain over greedy is large for a weak policy and shrinks toward zero as policy strength $\alpha(\pi)$ grows (RQ2); on a strong policy near the task’s accuracy ceiling, additional search is barely worth its cost.
3. **Prefer reward-guided search to sampling.** At matched budget, self-consistency—which ignores the reward—captures almost none of the gain that reward-guided MCTS does (RQ1).

The binding constraints are the *search space* and the *policy*, not the search budget or the reward source.

7.2 Cascade error: why taxonomy search hurts

Proposition 2 predicts that level-by-level commitment compounds early mistakes. The per-example breakdown (weak policy, 5 seeds $\times$ $n=300$, Figure 5) confirms it: **MCTS-Tax commits to a wrong category — unrecoverable by construction — on 60.5% of examples, versus 51.9% for MCTS-Reason** (greedy: 71.3%), and on the examples where MCTS-TAX chose the wrong category, MCTS-REASON recovered the correct one 25.7% of the time. Notably, the within-category error rate (right category, wrong leaf) is nearly constant across strategies (14–17%): all of the reasoning-search advantage comes from avoiding the early, coarse commitment, not from finer leaf discrimination. Deferring commitment is thus the mechanism behind reasoning search’s advantage.

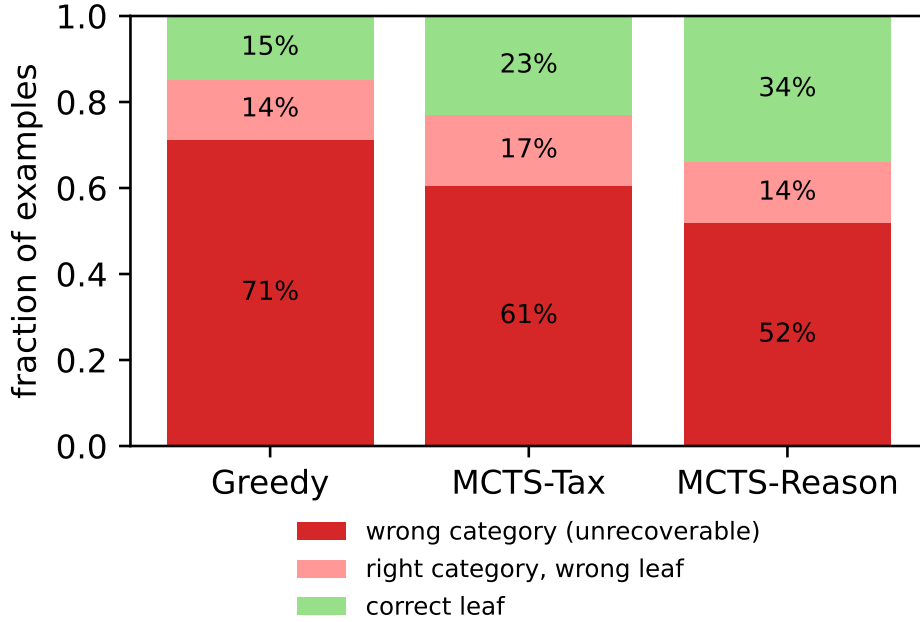


Figure 5: First-error level (GPT-4.1-nano, 5 seeds $\times$ 300 examples; greedy is deterministic, 1 seed). Strategies differ almost entirely in how often they commit to a wrong coarse category; the right-category-wrong-leaf band is nearly constant.

7.3 The scaffold, not the search

The MCTS-REASON budget sweep (Figure 4) adds a sharper reading of RQ4: accuracy at $I=2$ — where the “tree” has barely expanded beyond its root and the reward has had no opportunity to steer — already matches $I=8$. Most of the benefit of “MCTS over reasoning” is therefore attributable to its *scaffold* (generate free-form reasoning about the patch, then commit to a leaf directly, skipping level-by-level commitment), not to the tree-search machinery on top. This refines the headline: the search-space design dominates both the budget *and the amount of search itself*. By contrast, MCTS-TAX does benefit from iterations at small I (Figure 4), but saturates by $I \approx 8$ at a level far below the reasoning scaffold. The scaffold reading also explains the premium-policy reversal (RQ4): externalized step-by-step reasoning is a crutch the strongest policy no longer needs, so what remains useful there is only the taxonomy’s fine-grained discrimination.

7.4 Reward fidelity is not the bottleneck

Replacing self-evaluation with a stronger external judge (Claude-Haiku-4.5 judging GPT-4.1-nano) leaves taxonomy search exactly where it was (22.8 vs. 22.9 leaf) and lifts reasoning search by +1.5 points at +27% tokens — less than the same tokens buy via the search-space choice itself (RQ3). The asymmetry is informative: in MCTS-TAX the reward only re-ranks siblings *after* the category commitment that drives most errors (Figure 5), so no reward source can rescue it; in MCTS-REASON the reward selects among complete leaf commitments, where a better judge has something to act on — yet the effect is marginal because the policy’s own self-evaluation already separates its candidate commitments about as well.

The logged rollout rewards quantify this directly. Treating the reward of each MCTS rollout as a predictor of whether that rollout’s leaf is correct: on taxonomy rollouts the judge is *measurably better calibrated* than self-evaluation (AUROC 0.76 vs. 0.71, 14–24k pooled rollouts) — and final accuracy still does not move, confirming that reward quality is not what limits

MCTS-TAX. On reasoning rollouts the judge is *no* better calibrated (0.62 vs. 0.64), consistent with its marginal accuracy effect there. In both cases the reward ranks candidates far above chance, so the search is reward-guided in a meaningful sense; it is the set of candidates the search space exposes, not their ranking, that binds.

7.5 Latency and practical cost

Token budget understates MCTS’s practical cost: self-consistency parallelizes trivially across its independent samples, whereas MCTS is sequential (each iteration depends on the previous tree statistics). The flat MCTS-REASON iteration curve softens this in practice — at $I=2$ its sequential depth is small — but the asymmetry matters wherever more search *is* needed. **[TODO: Pareto frontier with wall-clock latency (P50/P95) on the x-axis.]**

7.6 Failure modes

The cascade breakdown above is the quantitative core of the failure analysis: taxonomy search fails predominantly by early-level commitment; reasoning search’s residual errors are split between wrong-category commits and within-category (sibling-CWE) confusions, the latter reflecting genuine taxonomy ambiguity and label noise (§7.7). **[TODO: Manual labeling of 100 errors into pillar-miss / sibling-miss / reward-misranking / ambiguous-ground-truth, weak vs. strong policy.]**

7.7 Limitations

- **Label noise and ambiguity.** BigVul CWE labels are known to be noisy, and many functions admit several valid CWEs while the gold label is single; this caps achievable accuracy independently of method (greedy category accuracy plateaus near 0.40 even for a premium model on raw functions).
- **Pilot scale.** Headline comparisons use $n=300$ (5 seeds) for the weak policy and $n=130\text{--}150$ (3 seeds) for DeepSeek/Haiku, all drawn from one fixed example file (the smaller sets are prefixes of the larger), with a few per-cell API failures dropped; the final protocol fixes a common $n\approx 2000$ test set with 5 seeds and paired McNemar throughout.
- **Saturation of MCTS-Tax.** We compare MCTS-TAX up to $I=16$ (10.7k tokens); deeper probes (I up to 128, full-depth tree, $n=8$) showed predictions frozen well before $I=16$, and MCTS-REASON at *half* MCTS-TAX’s budget already dominates (Figure 4), so larger taxonomy-search budgets are unlikely to change the conclusion.
- **Reward is zero-shot only.** A trained process reward model could shift the RQ3 conclusion; we leave it to future work.
- **Pretraining exposure.** We cannot fully rule out that LLMs saw test CVEs during pretraining; a post-cutoff slice is a planned robustness check.
- **Granularity coverage.** Findings are established on a 26-CWE subset of View-1003 (the labels occurring in the data) plus a floor probe at View-1000; the intermediate regime — the full 130-CWE View-1003 as used by NVD — is the natural target for the final protocol.
- **Language coverage.** Data is C/C++-dominated.

7.8 Tree granularity as a boundary condition

Does the picture survive at the full Research-Concepts granularity (View-1000, 716 leaves)? A probe ($n=50$) says the question itself dissolves there: the *strongest* policies drop to near-floor — Sonnet greedy reaches 20% pillar accuracy (10 pillars) and its predicted path passes through the gold node on only 6% of examples; MCTS-TAX ($I=16$), now spending 30k tokens/example, lifts this to 28% / 14%. Search still moves the needle (gold-on-path doubles), but from a base so low that no test-time budget makes the output usable. This is the boundary condition our policy-strength story predicts: search converts headroom into accuracy only when the policy is above the floor of the label space (RQ2), and at research granularity it is not — which is also why MITRE directs practical CVE mapping to the smaller View-1003 (§3). Test-time search is a lever *within* the operational regime, not a way to buy back a label space the policy cannot see.

[**TODO: Robustness: confirm the reasoning>taxonomy result on a temporal split (BigVul 2006–16 vs. 2017–19).**]

8 Conclusion

We presented a compute-matched study of test-time search for hierarchical CWE classification, holding the policy LLM and prompt fixed and comparing seven strategies at equal token budgets. By isolating policy strength and reward fidelity, we separate the value of *search structure* from the value of *extra compute* — a confound that pervades reported MCTS-for-LLM gains.

Searching a free-form reasoning trajectory beats searching the output taxonomy by +12 leaf points at *half* the token budget on the weak policy ($p < 10^{-4}$), and on four of five policies overall, because taxonomy search commits to coarse categories early and compounds the error—making the *search space* the primary design lever, above budget or reward. The margin closes, and at the premium end reverses, as the policy strengthens: a policy that reasons well unaided no longer needs the scaffold. Indeed, the reasoning strategy’s accuracy is flat in its iteration knob: the scaffold (reason, then commit to a leaf directly), not the tree search on top, carries the gain. At matched compute, reward-guided tree search also delivers a clear gain on a weak policy (+8.2 leaf points over greedy, +6.4 over self-consistency) but a negligible one on a strong policy (+1.9); self-consistency—sampling without a reward—helps neither. The marginal value of search thus *decreases* with policy strength, contradicting the intuition that better policies search better. The binding constraint is the accuracy ceiling of the policy on the task, not the search budget: once a policy nears that ceiling, additional search reconfirms or drifts. We read this as a budget-aware rule (§7): spend on search when the policy is weak and the reward is informative; spend on a better policy otherwise.

A budget-aware decision rule. Our main practical deliverable is a rule for *when* to spend on search: read off the policy’s pillar accuracy $\alpha(\pi)$ and the available budget $\mathcal{B}$, then choose greedy, self-consistency, or MCTS accordingly (§7).

Future work. A trained process reward model (PRM) could shift the reward-bottleneck conclusion; we left it out to keep the study GPU-free, and flag it as the most informative next experiment. Extending the protocol to multi-language data and to detection/localization (beyond classification) are natural follow-ups.

Broader Impact. LLM-assisted vulnerability triage is already deployed under tight cost budgets; our results give practitioners a calibrated answer to “is structured search worth its tokens here?”—often the answer is to fix the input representation and the search space before buying more compute, which lowers both cost and the energy footprint of security tooling. At the same time, automated CWE labels are a triage aid, not a verdict: our best pilot configuration

still mislabels most samples at the leaf level, and a false negative can hide a real weakness. Deployments should keep a human reviewer in the loop and treat predicted labels as prioritization signals rather than ground truth. The study uses only public, already-disclosed vulnerabilities and fixes, so it does not create new disclosure risk.

References

- [1] Guru Bhandari, Amara Naseer, and Leon Moonen. CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*, 2021.
- [2] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- [3] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(9):3280–3296, 2021.
- [4] Guoxin Chen, Minpeng Liao, Chengxi Li, and Kai Fan. Alphamath almost zero: Process supervision without process. *arXiv preprint arXiv:2405.03553*, 2024.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. Are more llm calls all you need? towards scaling laws of compound inference systems. *arXiv preprint arXiv:2403.02419*, 2024.
- [6] Yizheng Chen, Zhoujie Ding, Lamy Alowain, Xinyun Chen, and David Wagner. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2023.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [8] Siddhartha Shankar Das, Edoardo Serra, Mahantesh Halappanavar, Alex Pothén, and Ehab Al-Shaer. V2w-bert: A framework for effective hierarchical multiclass classification of software vulnerabilities. *arXiv preprint arXiv:2102.11498*, 2021.
- [9] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we? In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [10] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag. *arXiv preprint arXiv:2406.11147*, 2024.
- [11] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020.
- [12] Michael Fu and Chakkrit Tantithamthavorn. Linevul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, 2022.

- [13] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [14] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of EMNLP*, 2023.
- [15] Sihao Hu, Tiansheng Huang, Fatih İlhan, Selim Furkan Tekin, and Ling Liu. Large language model-powered smart contract vulnerability detection: New perspectives. *arXiv preprint arXiv:2310.01152*, 2023.
- [16] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- [17] Aris Kosmopoulos, Ioannis Partalas, Eric Gaussier, Georgios Paliouras, and Ion Androutsopoulos. Evaluation measures for hierarchical classification: A unified view and novel approaches. *Data Mining and Knowledge Discovery*, 29:820–865, 2015.
- [18] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.
- [19] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024.
- [20] Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lyna Zhang, Fan Yang, and Mao Yang. Mutual reasoning makes smaller llms stronger problem-solvers. *arXiv preprint arXiv:2408.06195*, 2024.
- [21] Carlos N. Silla and Alex A. Freitas. A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22:31–72, 2011.
- [22] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [23] Benedikt Stroebl, Sayash Kapoor, and Arvind Narayanan. Inference scaling flaws: The limits of llm resampling with imperfect verifiers. *arXiv preprint arXiv:2411.17501*, 2024.
- [24] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Miaolei Shi, and Yang Liu. Llm4vuln: A unified evaluation framework for decoupling and enhancing llms’ vulnerability reasoning. *arXiv preprint arXiv:2401.16185*, 2024.
- [25] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.
- [26] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [27] Zihan Wang, Peiyi Wang, Lianzhe Huang, Xin Sun, and Houfeng Wang. Incorporating hierarchy into text encoder: A contrastive learning approach for hierarchical text classification. In *Proceedings of ACL*, 2022.

- [28] Jonas Wehrmann, Ricardo Cerri, and Rodrigo Barros. Hierarchical multi-label classification networks. In *International Conference on Machine Learning (ICML)*, 2018.
- [29] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for llm problem-solving. In *International Conference on Learning Representations (ICLR)*, 2025.
- [30] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [31] Dan Zhang, Sining Zhou, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. Rest-mcts*: Llm self-training via process reward guided tree search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [32] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [33] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning (ICML)*, 2024.
- [34] Jie Zhou, Chunping Ma, Dingkun Long, Guangwei Xu, Ning Ding, Haoyu Zhang, Pengjun Xie, and Gongshen Liu. Hierarchy-aware global model for hierarchical text classification. In *Proceedings of ACL*, 2020.
- [35] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

A Dataset Construction Details

This appendix documents the data actually used in the pilot experiments (§6); steps that belong to the final protocol but are not yet exercised by the pilot are explicitly marked *planned*.

A.1 Source and input construction (pilot)

All pilot examples come from BigVul [11], which provides, for each vulnerability-fixing commit in C/C++ projects, the vulnerable function (`func_before`), the fixed function (`func_after`), the commit message, and a CVE-derived CWE label. We keep rows marked vulnerable whose CWE normalizes to a leaf of our tree (below), and render each example as a *unified diff* from the vulnerable to the fixed function, prefixed by an instruction comment (“’-’ lines are the VULNERABLE code ... classify the weakness in the removed/vulnerable code”) and the first 500 characters of the commit message. The diff localizes the vulnerable lines and is the input representation under which models clear the $\approx 40\%$ category ceiling we observed on raw functions. Diffs with fewer than three lines are dropped; code is truncated at 5,000 characters at prompt time (§B).

A.2 Splits (pilot)

The dev set ($n=50$) is drawn from the BigVul validation split and the pilot set ($n=500$) from the BigVul test split — so prompt iteration on dev cannot overfit the pilot — using per-CWE stratified round-robin sampling (seed 42). Pilot statistics: 500 examples, 18 distinct CWE leaves, 73 distinct projects; median diff length 15 lines (mean 27). Category composition: BUFFER 214, INJECTION 117, ACCESS 77, LIFETIME 54, NUMERIC 28, RESOURCE 5, CONTROL 5; the largest leaves are CWE-119 (128), CWE-20 (112), CWE-125 (68), CWE-200 (49). Most reported runs use the first 300 (nano) or 150 (Haiku, DeepSeek) examples of this fixed, shuffled file; per-example outputs are released for paired analyses.

A.3 CWE trees

Full tree. We download the official MITRE CWE corpus (`cwec_latest.xml`), extract the Research-Concepts view (View-1000) ChildOf hierarchy rooted at the ten Pillars, and resolve multi-parent nodes to a *primary* parent by breadth-first search from the pillars (first, i.e. minimal-depth, discovery wins). The result has 944 nodes and 716 leaves (depths 0–5) and is released as adjacency JSON with SHA-256 `7aeff209...26948a6`.

Coarse code-aligned tree (pilot). MITRE pillars are research abstractions (“Improper Control of a Resource Through its Lifetime”) that even strong policies place poorly, which floors level-0 accuracy and makes the policy-strength axis (RQ2) unreadable. For the pilot we therefore regroup the 22 dataset CWEs into a 2-level tree with 7 code-recognizable super-categories (spatial memory, temporal memory / lifetime, resource exhaustion, numeric, injection / input handling, control-flow, access control / info exposure / misc); leaves keep their MITRE names and descriptions. All 26 leaves are members of CWE View-1003, the simplified-mapping view that NVD uses to label published CVEs, so this label space is a subset of the official operational vocabulary rather than an ad-hoc simplification. The tree is released with SHA-256 `f3013c19...9c79aa`. The full-depth tree is used only for the granularity boundary-condition probe (§7.8).

A.4 Multi-source pooling and deduplication (planned)

The final protocol pools PrimeVul [9], DiverseVul [6], BigVul, and CVEfixes [1], with function-level MinHash deduplication (Jaccard ≥ 0.85) and repository-level disjointness across splits; the pilot uses a single source (BigVul) with splits inherited from the dataset’s own validation/test partition, so cross-source leakage is not yet a concern at pilot scale.

A.5 Pretraining-leakage slice (planned)

Because public CVEs may appear in pretraining corpora, the final protocol reports a slice restricted to CVEs published after each model’s training cutoff. At pilot scale we note that all four policies see the same examples, so leakage would shift absolute numbers rather than the strategy comparisons that are the object of study.

A.6 Datasheet (planned)

A Datasheets-for-Datasets [13] entry (motivation, composition, collection, preprocessing, uses, distribution, maintenance) will accompany the released benchmark bundle.

B Prompts and Schemas

All strategies share a single per-step policy prompt (§B.1); the reasoning-trajectory strategies add two prompts of their own (§B.2); both reward sources share one scoring prompt (§B.3). All calls request `response_format = json_object` and are parsed defensively: malformed entries are dropped, and any candidate missing from the output receives a floor score of 10^{-3} so search never crashes on a parse failure. Code longer than 5,000 characters is truncated with an explicit `/* ...truncated ...*/` marker.

B.1 Per-step ranking prompt (all taxonomy strategies)

Greedy, self-consistency, best-of- N , beam, and MCTS-TAX all call one shared step: rank the candidate children of the current taxonomy node. Placeholders are in angle brackets.

```
SYSTEM:
You are a security expert classifying the vulnerability in a code
function according to the CWE taxonomy. Classify the WEAKNESS the
code exhibits, not the fix. You are given the candidate CWE
categories at one level of the taxonomy; choose among THESE
candidates only.

USER:
[Prior reasoning: <rationale carried over from the parent step>]

Function:
'''c
<code, truncated at 5000 chars>
'''

Candidate CWE categories:
<one line per child: "- CWE-XXX: <name>. <description>">

Score EVERY candidate listed above by how likely it describes the
weakness in the function (0.0 = no, 1.0 = yes); scores need not
sum to 1. You MUST output an entry for every candidate CWE id.
Return JSON: {"scores": {"CWE-XXX": 0.0, "CWE-YYY": 0.0, ...},
"rationale": "one sentence"}.
```

Greedy takes the arg-max at temperature 0; self-consistency and best-of- N sample the same prompt at temperature 0.7 with a per-sample cache nonce; beam and MCTS-TAX use the full score vector as child priors. The `rationale` is capped at 300 characters and passed down as “prior reasoning” to the next level.

B.2 Reasoning-trajectory prompts (MCTS-Reason)

MCTS-REASON searches over reasoning states rather than taxonomy nodes. A move is either a brief next reasoning step or a commit to one leaf CWE; the candidate list shown to the model is the flat list of tree leaves (id + name).

```
SYSTEM:
You are a security expert reasoning step by step about the weakness
in a code patch, in order to classify it into exactly one CWE.
Reason about the removed (vulnerable) lines, not the fix.

USER (expansion; k = 3):
Code patch:
'''c
```

```

<code, truncated at 5000 chars>
'''

Reasoning so far:
<numbered steps, or "(none yet)">

Candidate CWEs:
<one line per leaf: "- CWE-XXX: <name>">

Propose 3 DISTINCT next moves toward the answer. Each move is
either a brief next reasoning step, or a final commit to ONE
candidate CWE id. Return JSON: {"moves": [{"type": "step",
"text": "..."} or {"type": "commit", "cwe": "CWE-XXX"}, ...]}.

USER (forced commit; rollouts and final extraction):
<same code + reasoning-so-far + candidate blocks>

Commit to the single best CWE now. Return JSON {"cwe": "CWE-XXX"}.

```

Reasoning steps are capped at 300 characters and traces at `max_steps = 3`; a commit to an id outside the candidate list falls back to the first leaf (and is counted as an error in practice). Expansion runs at temperature 0; rollout commits run at temperature 0.7 with a (seed, iteration) cache nonce.

B.3 Reward prompt (self-evaluation and external judge)

Both reward sources use the *same* prompt; the only difference is the model it is sent to — the policy itself (`self_eval`) or the stronger judge (`external_judge`, Claude-Haiku-4.5). The reward scores a partial or complete root-to-node path in $[0, 1]$; both are zero-shot, no PRM is trained.

```

SYSTEM:
You are auditing a proposed CWE classification of a code function.
Judge how well the proposed CWE path describes the weakness in the
code. Classify the weakness, not the fix.

USER:
Function:
'''c
<code, truncated at 5000 chars>
'''

Proposed CWE path: CWE-AAA (<name>) -> CWE-BBB (<name>) -> ...
This path is {PARTIAL (not yet a leaf) | COMPLETE (leaf)}.

Description of the final node CWE-BBB: <description or "n/a">

Return JSON {"score": 0.0-1.0} where score is the probability this
path correctly classifies the weakness in the function.

```

The score is clamped to $[0, 1]$; a parse failure scores 0.

C Additional Results

C.1 Full pilot budget grid (weak policy)

Table 4 reports every (strategy, knob) point behind Figure 4: GPT-4.1-nano, $n=300$, mean $\pm$ std over 5 seeds (greedy is deterministic and run once).

Strategy	Knob	Tokens/ex	Leaf (%)	Category (%)
Greedy	—	1,561	14.7	28.7
Self-consistency	$N=4$	6,246	16.6 ± 0.4	31.8 ± 1.0
Self-consistency	$N=8$	12,493	16.5 ± 1.1	31.5 ± 1.0
Best-of- N	$N=8$	15,836	21.1	39.5
Beam	$w=4$	3,753	17.0	35.0
MCTS-TAX	$I=8$	6,695	22.3 ± 0.1	40.2 ± 0.4
MCTS-TAX	$I=16$	10,669	22.9 ± 0.1	39.5 ± 0.4
MCTS-REASON	$I=2$	5,257	34.8 ± 0.4	50.4 ± 0.4
MCTS-REASON	$I=4$	8,546	34.5 ± 0.6	49.1 ± 0.9
MCTS-REASON	$I=6$	11,379	33.8 ± 0.4	48.7 ± 0.4
MCTS-REASON	$I=8$	14,142	33.9 ± 0.3	48.1 ± 0.3

Table 4: Pilot budget grid (GPT-4.1-nano, BigVul diff+msg, $n=300$, 5 seeds; best-of- N and beam: 1 seed, best-of- N $n=289$ after API failures). Tokens/ex doubles as the knob $\rightarrow$ tokens calibration used to set compute-matched comparison points.

Across-seed variance is small throughout (std ≤ 1.1 points), so the pilot’s seed budget is adequate for the gaps being measured.

C.2 Pairwise significance (weak policy)

Paired McNemar on leaf accuracy, seed 13, $n=300$ (consistent across seeds): MCTS-REASON($I=8$) vs. MCTS-TAX($I=16$): 57/24 discordant pairs, $p = 3.2 \times 10^{-4}$; MCTS-REASON($I=2$) vs. MCTS-TAX($I=16$) (at *half* the budget): 59/23, $p = 8.7 \times 10^{-5}$; MCTS-REASON($I=2$) vs. greedy: 80/19, $p = 4.4 \times 10^{-10}$; MCTS-TAX($I=16$) vs. greedy: $p = 8 \times 10^{-6}$; MCTS-TAX($I=16$) vs. self-consistency ($N=8$): $p = 2 \times 10^{-3}$. The full strategy $\times$ strategy matrix with Bonferroni adjustment is part of the final protocol.

C.3 Taxonomy-search saturation probe

On the full-depth MITRE tree (716 leaves), MCTS-TAX at $I \in \{16, 64, 128\}$ returned byte-identical predictions on a small dev probe ($n=8$; 13k, 38k, and 71k tokens/example respectively): once the level-wise priors lock in, more iterations re-visit the same branch. This motivated capping MCTS-TAX at $I=16$ in the pilot grid.

C.4 Planned ablations (final protocol)

The following are pre-registered but not yet run at pilot scale: exploration-constant sensitivity ($c_{\text{exp}} \in \{0.5, 1.0, 1.41\}$); aggregation rule (majority vs. reward-weighted vs. robust-child); tree-granularity breakdown (coarse vs. full-depth); per-language stratification (the pilot is C/C++ only); wall-clock latency tables (P50/P95) separating parallelizable from sequential token spend.